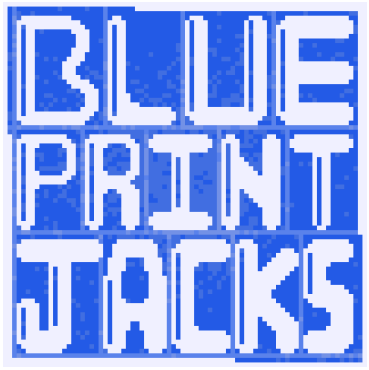
Tech Feasibility

October 24, 2025

PROJECT SPONSOR

- **Chris Ortiz** Senior Technologist, Tools & Infrastructure
- **Rex Jackson** Vice President, Tools & Infrastructure

TEAM DETAILS



- Team Name: **Blueprint Jacks**
- Team's Faculty Mentor: **Ogonna Eli**
- Team Members: **Jackson Belzer**, **Christian Lamb**, **Juan José Serrano Mora**, **Alejandro Benito Marcos**

Table of Contents

1. Introduction

1.1. The Big Picture

In this day and age of technology, the reliability and quality of complex systems has never been more critical. From the smartphones we use every hour to medical devices and autonomous vehicles that must operate perfectly every second, one design flaw in these systems could quickly lead to catastrophic consequences, including total data corruption, irreparable harm to a company's reputation, and even lives lost.

At SanDisk, a global leader in flash storage solutions, verifying the correctness of designs for storage systems is crucial as errors can replicate across millions of devices and impact customers worldwide. As a consequence, the company has put a significant investment into thorough design processes, collaborative engineering practices to minimize errors, and meet the expectations of a competitive, high stress market.

1.2. The Problem

Currently, design documents at SanDisk are written in plain English and supplemented with diagrams, tables, and pseudocode. They are subsequently reviewed iteratively by various engineers during design reviews to attempt to find errors. While this process is standard practice in many fields and engineering firms, it is limited by nature: subtle design errors, particularly those involving concurrency or nondeterminism, can easily go unnoticed. Also, creating the corresponding visual figures manually is time consuming and labor intensive, further slowing down the design review process.

To address the shortcomings of this workflow, SanDisk has begun exploring formal methods, specifically TLA+ (Temporal Logic of Actions), a mathematical specification language to model, reason about, and verify system behaviors with precision and rigor. TLA+ allows engineers to explore all possible states and execution paths of a system, ensuring that even rare concurrent conditions are checked automatically before implementation, thanks to the TLA+ model checker (TLC), which analyzes a "model" (this is what we create with TLA+), an abstract mathematical representation of the system's states and the actions that change them. However, writing TLA+ directly can be challenging for engineers accustomed to pseudocode and visual diagrams. To bridge this gap, the company has adopted PlusCal, a pseudocode language that can be automatically translated into TLA+, enabling engineers to write specifications in a familiar form while still benefiting from the formal verification that TLA+ provides.

1.3. Our Solution Vision

Our vision is to develop the P2P2P (PlusCal to PlantUML to PDF) tool, a seamless system that turns formally verified design specifications, written in PlusCal, into clear, professional documentation automatically. Instead of manually translating technical designs into diagrams and reports, our tool takes the PlusCal pseudocode, which has already been verified for correctness with the TLC, and generates visual diagrams (flowcharts, UML sequence diagrams, and activity diagrams) and locked PDF

documents ready for distribution. Key features would include:

- **Automatic Diagram Generation:** Transforms PlusCal code into flowcharts, UML sequence, and activity diagrams automatically, saving time and reducing manual errors.
- **Formal Verification and Accuracy:** Guarantees that all diagrams accurately reflect the behavior verified by the TLC.
- **Efficient Workflow:** Integrates seamlessly into the development environment, allowing engineers to produce diagrams and PDFs without leaving their tools.
- **Reliable Final Documentation:** Generates professional, locked PDFs that stakeholders can trust to match formally verified designs.

1.4. Impact

This breakthrough will benefit our client in three important ways:

1. It will reduce the time required to create visual figures while ensuring they are formally verified, giving stakeholders greater confidence in the design documents.
2. It will encourage firmware engineers to adopt PlusCal as a stepping stone to TLA+, lowering the barrier to formal verification and improving design quality.
3. Automated diagram generation will promote a culture of formal methods within SanDisk's engineering processes, streamlining collaboration and reducing errors.

By automating the transformation from verified PlusCal code to professional documentation, our solution addresses the inefficiencies and labor intensity of the current process. Engineers can focus on innovation, design reviews will be faster and more reliable, and cross team communication will be clearer. The result is a more agile, confident, and quality driven engineering workflow, a mission ready to be accomplished.

2. Technological Challenges

2.1. Introduction

Building the P2P2P(PusCal to PlantUML to PDF) tool brings forth several technological challenges that will need to be overcome if we want an automated, robust, and usable workflow. Each of the steps of the process, which are verification, diagram creation, and documentation, need to be connected properly between formal methods and engineering tools. The key objective should be to enable engineers to move from verified PlusCal code to synchronized professional-quality documentation with minimal manual work.

The following subsections explain each of these challenges in more detail. Section 2.2 focuses on automating diagram generation from PlusCal code and ensuring accuracy and synchronization with verified models. Section 2.3 considers maintaining formal correctness and consistency between the generated diagrams and TLC outputs, and Section 2.4 tackles workflow efficiency and how to structure multiple tools into one automated pipeline. Finally, 2.5 explains how to produce a reliable locked PDF output that ties verified content together for review. These challenges together comprise the essential technical aims needed to carry out a secure and scalable solution to SanDisk's design verification workflow.

2.2. Automatic Diagram Generation

- **Challenge:** Convert PlusCal code into UML sequence diagrams, activity diagrams, and flowcharts automatically, ensuring diagrams remain synchronized with the verified code. Manual diagram creation is time consuming and prone to errors, so automation is essential.
- **Details:**
  - Accuracy: Diagrams must fully reflect the verified behavior of PlusCal code.
  - Ease of Use: Engineers with limited TLA+ experience should be able to generate diagrams with minimal effort.
  - Compatibility: Works seamlessly with PlantUML and LaTeX for PDF generation.
  - Speed: Generation time should be under one minute per diagram.

2.3. Formal Verification and Accuracy

- **Challenge:** The generated diagrams must exactly reflect the behavior verified by the TLC. This requires testing models with the TLC before any diagram is generated. Any mismatch could lead to incorrect design review decisions.
- **Details:**

- ➢ Ease of development: Using command line parsing of the TLC output simplifies implementation and reduces the need for deep modification of existing VSCode/TLC extensions.
- ➢ Maintainability: Minimal dependencies on TLC internals; future updates require only checking for output format changes.
- ➢ Feasibility: Fits the team's current skillset in Rust/Python and project timeline.
- ➢ Minimally intrusive: Engineers continue their usual workflow; the tool automatically triggers diagram generation after successful verification.

2.4. Efficient Workflow

- ❖ **Challenge:** Reduce reliance on manual processes, automate repetitive tasks, and provide a seamless end to end workflow from PlusCal verification to diagram and PDF generation.
- ❖ **Details:**
  - ➢ Automation: VSCode extension triggers verification, diagram generation, and PDF compilation with a single command.
  - ➢ Integration: Rust backend integrates PlusCal parsing, PlantUML diagram creation, and LaTeX PDF generation into a single pipeline.
  - ➢ Speed: Local execution and elimination of manual file handling ensures diagrams and PDFs are generated in seconds.
  - ➢ Ease of Use: Engineers interact with a single command inside VSCode; no need to switch tools or handle intermediate files.
  - ➢ Scalability: Pipeline handles large and complex models efficiently, maintaining performance and stability.

2.5. Reliable Final Documentation

- ❖ **Challenge:** Produce professional, locked PDFs that combine verified code and diagrams, ensuring consistency and reproducibility for stakeholders.
- ❖ **Details:**
  - ➢ Automation: LaTeX compiles all diagrams and text into a final PDF with one command.
  - ➢ Integration: PlantUML diagrams automatically update in the PDF, maintaining synchronization with verified PlusCal code.
  - ➢ Version Control: Entire workflow, including LaTeX sources and diagrams, is Git friendly for tracking and collaboration.
  - ➢ Cross Platform: Works on Windows, macOS, and Linux with minimal setup.
  - ➢ Ease of Learning: The team can quickly adopt LaTeX templates with existing documentation and examples.
  - ➢ Styling Control: LaTeX ensures professional, consistent formatting and flexible customization.
  - ➢ Build Speed: PDFs compile in under a minute, enabling fast iteration and review.

3. Technology Analysis

3.1. Introduction

Before starting with development, it is important to make sure our project can actually be built with the tools, time, and skills that we have. This section explains how we tested the technological feasibility of the P2P2P system. It identifies the main challenges our team faced, brings up possible solutions, and explains why we chose the specific methods that we did. By taking this into account, we are showing that our design choices are researched and ready for implementation.

3.2. Automatic Diagram Generation

- **Introduce the Issue:**

  Engineers currently create diagrams manually, which is time consuming and prone to errors. The lack of tools for this matter makes it even harder to test and create the proper diagrams to ensure a correct workflow. We need a tool that automatically transforms PlusCal code into UML diagrams and flowcharts in order to simplify this process and avoid doubling the work.

- **Desired Characteristics:**

  - Complete accuracy of the diagram with respect to the verified code.
  - Easy to maintain and use by engineers with limited TLA+ experience.
  - Compatible with PlantUML and PDF generation.
  - Fast generation time (< 1 minute per design).

- **Alternatives:**

  - **1. Custom Rust Script (PlusCal → PlantUML)**

    Write a custom Rust program that parses PlusCal code and automatically generates PlantUML diagrams. This approach gives full control over the conversion logic and ensures diagrams accurately represent the verified PlusCal code. It integrates well with the planned VSCode workflow but requires development and maintenance effort. It consists of the following phases: Lexical (token recognition), Grammatical (token sequence recognition), Semantical (context recognition) and finally, code generation.

  - **2. Existing TLA+ → UML conversion tools (non-existing yet)**

    This would use any existing tools capable of converting TLA+ or PlusCal to UML diagrams automatically. Currently, no known tools exist for this

functionality, so while this could theoretically save development time, it is not a practical option at the moment.

➢ **3. External DSL → PlantUML**

Transform PlusCal code into an intermediate Domain Specific Language (DSL) such as JSON or DOT, then generate PlantUML diagrams from this intermediate representation. This separates the parsing from diagram generation, making it modular and easier to maintain. However, it adds complexity to the workflow and introduces an additional processing step.

❖ **Analysis:**

The following analysis evaluates the three alternatives for automatic diagram generation based on key criteria: ease of development, maintainability, accuracy, feasibility, and minimal intrusion into the engineers' workflow. Each criterion is discussed for every alternative to provide a clear rationale behind the table ratings.

❖ **Custom Rust Script (PlusCal → PlantUML)**
- ➢ **Ease of Development:** Developing a Rust parser requires building lexical, grammatical, and semantic processing stages, but the team has sufficient previous experience, making implementation realistic.
- ➢ **Maintainability:** Once developed, the script is modular and maintainable; updates to PlusCal syntax may require adjustments, but these are straightforward to implement.
- ➢ **Accuracy:** Full control over the parsing logic ensures diagrams accurately reflect the verified PlusCal code.
- ➢ **Feasibility:** This approach is highly feasible given the team's skills, timeline, and available tools.
- ➢ **Minimal Intrusion:** The tool can integrate smoothly into the VSCode workflow, requiring minimal extra steps from engineers.

❖ **External DSL → PlantUML**
- ➢ **Ease of Development:** Developing a DSL intermediate layer adds complexity; it requires defining the DSL, writing translators, and managing two conversion steps.
- ➢ **Maintainability:** The two step process increases maintenance overhead, as both the DSL translator and PlantUML generator must be updated if PlusCal changes.
- ➢ **Accuracy:** Accuracy is high if the intermediate representation is well defined, but there is potential for subtle errors during translation.

- ➢ **Feasibility:** Technically feasible, but more time consuming than the Rust parser approach.
- ➢ **Minimal Intrusion:** Requires engineers to trust an extra processing step, which slightly complicates the workflow.

❖ **Chosen Approach:**

Custom Rust Script (PlusCal → PlantUML)

We selected the Custom Rust Script because it ensures complete accuracy with respect to verified PlusCal code, is feasible within the project timeline, and can be integrated into the VSCode based workflow with minimal disruption. This approach also allows future improvements or adaptations without dependency on non-existent external tools.

Comparison Chart:

Alternative	Ease of Development	Maintainability	Accuracy	Feasibility	Minimal Intrusion
Custom Rust Script (PlusCal → PlantUML)	⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
External DSL → PlantUML	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐

Table 1: Comparing performance of alternatives for Automatic Diagram Generation in various categories

The Rust script approach provides the best combination of accuracy, feasibility, and maintainability. While the DSL option is modular, it introduces an extra layer of complexity that could slow development. Existing tools, although theoretically ideal, do not exist and therefore cannot be relied upon. Overall, the Rust script balances full control over diagram generation with a manageable development effort and smooth integration into the existing workflow.

❖ **Proving Feasibility:**

To validate this approach, we have a compiler using JavaCC for a simpler language called GCL, which follows the same processing steps planned for the Rust parser: lexical analysis (token recognition), grammatical parsing (token sequence recognition), semantic analysis (context recognition), and code generation. This simpler (yet complex) example demonstrated that the parsing to diagram workflow works correctly in practice. Using the lessons learned from

the GCL compiler, we are confident that the Rust parser for PlusCal can reliably generate PlantUML diagrams. Once verified, this pipeline will be expanded to handle the full workflow, demonstrating that automated diagram generation is feasible, accurate, and practical for real world engineering scenarios.

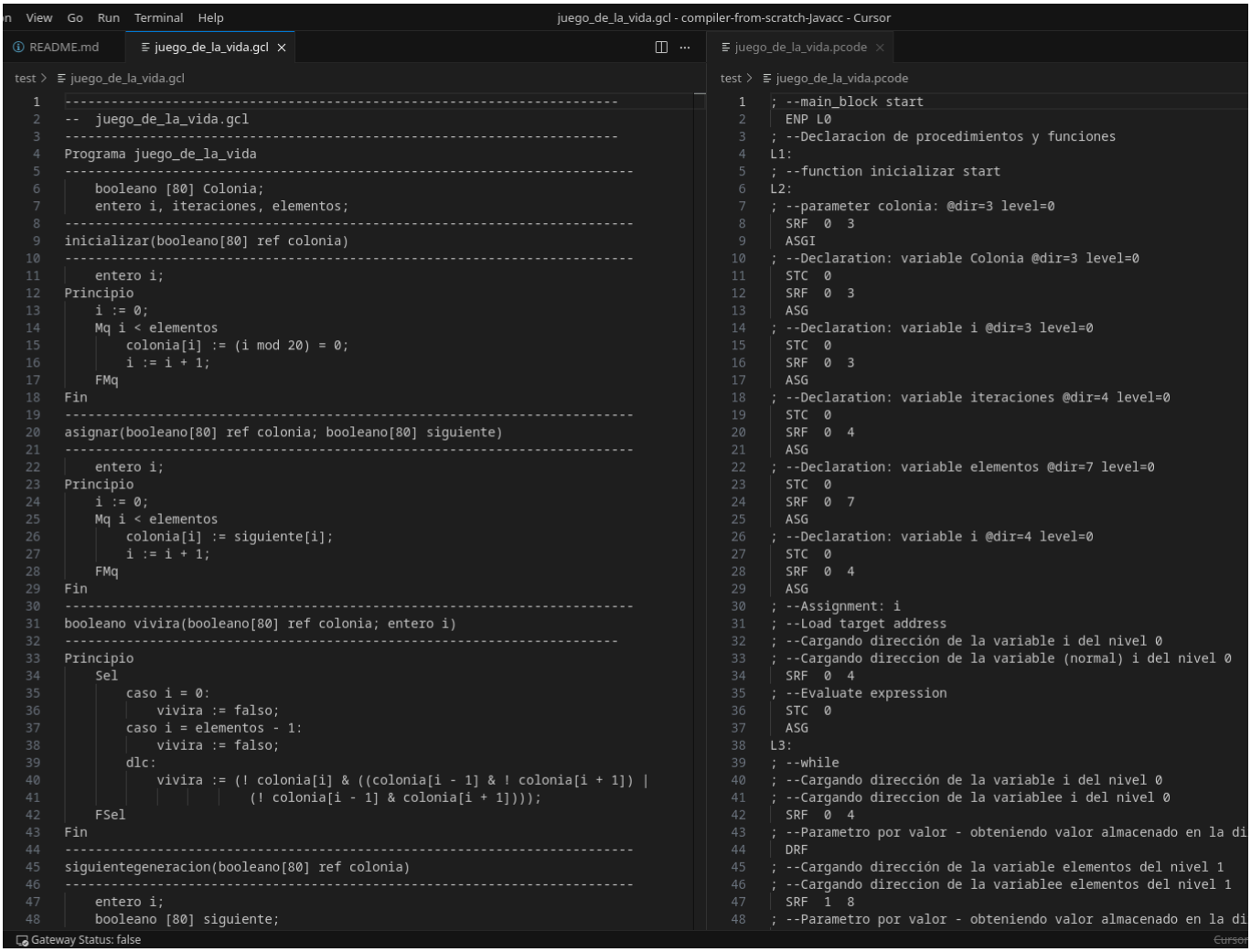


Figure 1: GCL language translated to assembly

[compiler-from-scratch-javacc]

3.3. Formal Verification and Accuracy

- **Introduce the Issue:**

  Engineers and professionals that are tasked with reviewing activity diagrams have to make sure that they are both correct and accurately represent the design of the system in TLA+ and they can never be sure that they have covered every possible edge case or issue that might come up. When checking their models with the TLC, their model is automatically verified for correctness, and we only want to generate a model given the model accurately passes this test. There are a lot of different ways to identify this output characteristic, and we must determine which one is best.

- **Desired Characteristics:**

  - **Ease of development**
    The solution minimizes the amount of time needed to both research and develop the solution. The deadline for the end of the school year makes it so that we likely cannot fully understand the TLA+ codebase because it is about 361,949 lines of code long and has been in development for over a decade, while also completing the P2P2P project, making time an important consideration.
  - **Maintainability**
    The solution can last a long time, where future costs and time required to maintain the software is minimized. It is important that the tool is able to stay functional without much future maintenance because it would disrupt the work of engineers to have to wait on fixes, and waste company resources fixing the tool.
  - **Feasibility**
    The solution is something that is easy to implement, both based on the skill of the developers and the amount of time allotted for development. The team does not have much of an understanding in the VSCode extension running the TLC compared to someone who has previous experience in systems of this scale, or those that have already contributed to the project, so the solution must be reasonable given the team's skill, experience, knowledge, and time constraints.
  - **Minimally intrusive**
    The solution should fit easily into the workflow of the user and require the least amount of extra work for them to achieve the extra utility being offered. It is important that engineers find it easy to interact with this tool since it will be an integral part of their workflow, and we want to minimize integration costs that come from moving towards our solution.

- **Alternatives:**

- **1. Building Directly into the TLC:**

  Integrate the tool directly into the TLC, so that once the model passes, the option to generate a diagram is given.
  Inspired by the already existing interface of the TLC, it seemed possible to add extra buttons or other methods of interaction to the already existing basic layout that could interact with our tool. This would require modifying the existing VSCode Extension that handles the TLC. Where there are buttons that are given to the user when using the TLC, we would have to create our own custom option and offer it up for the user to generate a diagram once the model has passed. This would have quite a few challenges, like parsing through the open source code regarding the project and identifying the parts of the code that needs to be modified to add our tool into the pipeline. This would also incur the technical debt of needing to keep the tool up to date with future changes and updates that might come with the VSCode extension, or risk falling behind on functionality compared to the main build if maintenance is put off.

- **2. Using Command Line Output to Determine Validity:**

  Automatically generate a model upon TLC passing based on the output of the TLC. This solution would be significantly simpler, requiring only the program to take the output of the TLC and parse it for the required lines that show there were no errors with the model, then based on that output it would be able to either continue on with generating the model, or give feedback to the user that it was not able to generate due to the model not passing. This solution was found by working with the TLC and looking at the text output that it produces as a result of running and seeing that it says when the model has passed with the line "Model checking completed. No error has been found.". This would save on the technical debt that would be brought on by directly modifying the source code of the VSCode extension, while potentially being more disruptive to the workflow of the user due to it not being directly integrated with existing tools.

- **3. Adding a Custom Flag For Generating the Model:**

  Add generating a model as a command line argument like the " dump dot" flag for generating dot files.
  Currently, there is a diagram that can be generated with the TLC that shows every possible state of the machine and every possible path that the model could take to get there. The TLC is told to create this file based on a setting given to the TLC and is built directly into its execution, which is potentially something that our solution could also do. This type of

diagram is different because our goal is not to show every possible state, and way to get there, but instead show the general structure of the model. This solution would be similar to option 1, though with more control for the user, and potentially less source code needing to be parsed in order to get the solution working. This would still require the need to keep up with updates made to the main project and make sure that our tool stays working and stays up to date with new changes.

- **Analysis:**

  - **Ease of development.**
    Ease of development was tested by doing research on the scope of modifications or development that would be needed to accomplish the task with each approach. In particular for options 1 and 3, the source code was looked through to determine the amount of code it seemed would have to be changed or adjusted, and the time that it took to find given sections of the code was also considered when looking at how easy each approach would be.

    - **1. Building Directly into the TLC**: Giving the user the option to generate a diagram would require adding a user interaction and have that interaction conditional on whether the model passed or not. This type of interaction does not currently exist in the TLC and would have to be created by us, since every field that currently exists is there regardless of whether the model passes or not. That button would then need to connect to our software feature. Given the need to fully investigate the source code as it exists now, create new interactions, and connect the software, this option will score relatively low in this category.

    - **2. Using Command Line Output to Determine Validity**: This would be the simplest of the options, and would require the least amount of overhead research and development seeing as the only needed work would be to read the output from the TLC, and parse through that information to determine if it passes or not, which is relatively similar to reading a text file. This scores it highly in this category.

    - **3. Adding a Custom Flag For Generating the Model**: This would run into a similar amount of hurdles as option 1, though it would require less source code to be looked through, and some of the functionality likely already exists in the source code through the earlier mentioned " dump dot" flag. This means that while it still has some of the same problems as before, it will score higher than option 1, but lower than option 2.

- **Maintainability.**
  Maintainability would have to be tested based on how much it would depend on existing architecture, how changes to the core TLC might affect the program, and how likely those changes are to happen. For options 1 and 3, after looking through the source code and identifying the sections that would have to be further investigated, this would include a combination of considerations like how often that section of code has been changed and what other parts of code are dependent on that section, along with if changes would be easy to remedy. For option 2, it would look more narrowly at how the output from a successful and failing output from the TLC has been changed, and if modifying the program to work with changed outputs would be difficult.

    - **1. Building Directly into the TLC**: Creating a custom version of an open source software while maintaining modern features would require monitoring changes made to the main project, which seem to be made every 1-3 days, and merging those changes with our project and resolving any conflicts that might come about. While for the most part the software seems to be in a state where very few major changes happen that modify large sections or functionality of the project, it would still require continuous support when changes do happen, and those are more likely to happen since it's dependent on the UI of the extension.

    - **2. Using Command Line Output to Determine Validity**: This would only require monitoring whether the specific output lines that we are checking for are changed in any updates to the TLC, and updating the corresponding lines in our program if that does happen. Since the last time the success text was changed in the source code was 12 years ago, it is likely that this line will not change in the future, so the tool should be reasonably robust for the foreseeable future.

    - **3. Adding a Custom Flag For Generating the Model**: Inheriting many problems from option 1, this option would require updating and maintaining the new version of the open source project, but it would probably be slightly less maintenance since the command arguments seem to be a more isolated part of the program that is more willing to be changed and added to without needing to be maintained with the rest of the software.

- **Feasibility.**
  Feasibility would have to have a subjective look at the team's abilities, and the time constraints put on the project to determine if a given solution

makes sense at this scale, or would be reasonable to expect. For options 1 and 3, these considerations would have to look at the scale of projects our team has worked on before, skills in the given tools, history in modifying open source software, and the scale of changes needed based on prior considerations. The considerations for option 2 would look at a smaller subset of skills, which more has to do with parsing input from another program.

- **1. Building Directly into the TLC**: Given the experience of the developers on the project, the scope of the open source software we would be modifying, and the likely issues that will happen along the way, this will score lowly in feasibility when compared to option 2 and slightly below option 3.

- **2. Using Command Line Output to Determine Validity**: This would be the easiest option, and will score highly. Every developer on this project has experience with parsing output similar to what is expected since it is relatively basic, and even if they need refreshing, it is a simple skill to learn, and the need for updates is unlikely. This will score the best in this category.

- **3. Adding a Custom Flag For Generating the Model**: This would be a more reasonable thing to achieve in the project, but it would still carry along with it the baggage of needing to maintain updates, and the scope of parsing through the amount of command arguments that exist currently would be a decent undertaking. This makes it less feasible than option 2, but more feasible than option 1.

➢ **Minimally intrusive.**
Being minimally intrusive would be based on how easily each solution would fit into someone's workflow with the TLC. This would require identifying where in the workflow it would be, how much the current workflow would have to change to accommodate the changes, and how easy it is to identify how the tool is used.

- **1. Building Directly into the TLC**: This would be the least intrusive option. It would feel like a feature update in any other program or game where it's a new thing on top of the same tool and they will see a new option when they are doing what they normally would and can try it out. This would make it so that it scores the highest in this category.

- **2. Using Command Line Output to Determine Validity**: This would likely be the most intrusive because it would require

running a separate program that takes the output from the TLC. Ideally, they would only run it when they specifically want a diagram, but it might be the case where they finally get a passing test and want to see the diagram and now have to go out of their way to do it. This will score it the lowest.

- **3. Adding a Custom Flag For Generating the Model**: This will also fit directly within the tools needed to use the TLC, but it will be a little more out of the way since it is in the command line flags that have to be modified within the settings. In theory, though, an engineer could keep this flag always set as active within the arguments that are always run with the TLC and leave it there, though this might cost them time. So while it is possible to require minimal work from the engineer, it requires knowing about the flag in the first place and it isn't introduced to the user in a natural way. This will score it above option 2, but below option 1.

❖ **Chosen Approach:**

Using Command Line Output to Determine Validity

Creating a program that uses command line output to signal whether to run the program or not is the best option for the project.
Option 1 has the advantage of being very easy for the user to use and integrate into already existing workflows, but it falls short when it comes to the actual technical challenges and the costs required for maintaining the project.
Option 2, while being a little less intuitive for the user and requiring some costs for teaching engineers how to use it, has the best outlook when it comes to how easy it would be to make the software to identify whether or not the model passed the tests and the least ongoing maintenance required to keep it running.
Option 3 tries to split the difference and be the best of both worlds, becoming slightly easier to make and slightly more intrusive to the engineer experience, but the strides it makes in being a little easier to create is not enough to close the gap and bring it on par with option 2.

Comparison Chart:

Alternative	**Ease of development**	**Maintainability**	**Feasibility**	**Minimally intrusive**
Building Directly into the TLC	⭐⭐	⭐⭐	⭐⭐	⭐⭐⭐⭐⭐
Using Command Line Output to Determine Validity	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐

Adding a Custom Flag For Generating the Model	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐

Table 2: Comparing performance of alternatives for Formal Verification and Accuracy in various categories

- **Proving Feasibility:**

To prove how easy this would be to implement, I have created a small demo program that prints out the same line as the TLC when a model passes ("Model checking completed. No error has been found.") when given the option "pass",but it won't when given anything else. We then pass that output to a second program that will check each line for the passing line, and if it doesn't find the line, it will say it failed. That is demonstrated here, showing that parsing output of a program is feasible for changing future behavior of a program.

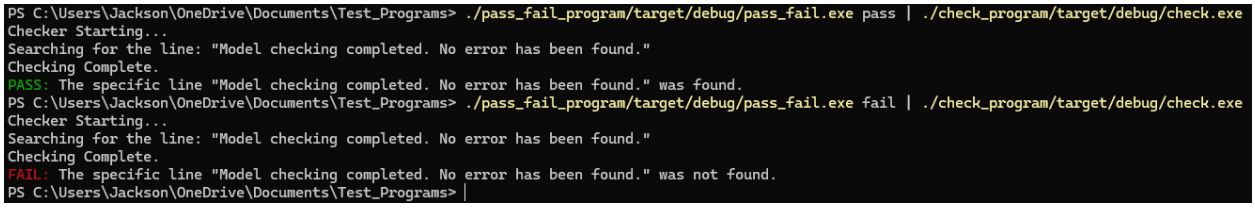


Figure 2: A program either produces or does not produce the line "Model checking completed. No error has been found." with pass or fail parameters respectively. This text then gets passed into a separate checking program that scans the output for that line.

3.4. Efficient Workflow

- **Introduce the Issue:**

  Currently, SanDisk's design verification workflow is highly manual and time consuming. Engineers must verify the specifications thoroughly during design reviews, generate diagrams by hand, ensure consistency between the specification and the visual artifacts, and lastly compile the verified content into a PDF. Each of these steps requires engineers to switch between different tools and file formats, slowing the design iteration down and producing risk of human error. An improved workflow should seamlessly connect formal verification, diagram generation, and documentation into a single automated pipeline that fits perfectly to the Product Designers' Design Process Flow.

- **Desired Characteristics:**

  - **Automation.**
    The workflow should minimize human intervention throughout the entire process, from verifying PlusCal specifications to generating the final PDF documentation. Once the engineer verifies the model, all subsequent steps (diagram creation, formatting, and PDF generation) should occur automatically, ensuring consistency and reducing human error.
  - **Integration.**
    The system must support seamless interaction between all tools involved: TLA+ for formal verification, PlantUML for diagram generation, and LaTeX for professional PDF documentation. This integration should eliminate the need for manual file transfers or intermediate steps, maintaining synchronization across every component of the pipeline.
  - **Speed.**
    The solution should produce verified diagrams and documentation within seconds rather than hours, allowing engineers to iterate quickly during the design and review process. Rapid feedback cycles are critical for maintaining productivity in fast paced development environments.
  - **Ease of Use.**
    The workflow should be intuitive, requiring minimal training. Engineers can generate diagrams and PDFs directly within their IDE with a single command, without manually handling files or switching tools.
  - **Scalability.**
    The system should efficiently handle large or complex design specifications without significant performance degradation. It must remain responsive even as project size, code complexity, or diagram detail increases, ensuring consistent performance across small and enterprise scale designs.

- **Alternatives:**

  - **1. VSCode Extension vs CLI Tool:** Implement the automation as a VSCode extension integrated into the engineers' existing TLA+ workflow, or as a standalone command line interface (CLI) tool. The VSCode extension provides a seamless user experience, allowing engineers to trigger diagram generation and PDF compilation directly from the IDE. The CLI is simpler to implement but requires manual execution outside the IDE, fragmenting the workflow and increasing the risk of errors.

  - **2. Integrated Pipeline vs External Scripts:** Determine whether to link the complete process: parsing PlusCal, producing diagrams in PlantUML, and compiling the PDF in LaTeX into a single automated file, or to run each step as a simple external script. Linking everything together achieves consistent outputs, reduces dependency problems, and improves reproducibility which is especially desirable for CI/CD and larger designs. External scripts are easier to get working initially, but might cause errors, need manual intervention, and convolute checking and execution of versions.

Note: These two decisions are related but distinct. The VSCode extension controls the user interface, while the pipeline integration determines how the workflow executes internally. Ideally, the extension will serve as a front end for a fully integrated pipeline, combining a smooth user experience with reliable automation.

- **Analysis**

  Each proposed solution was evaluated based on the desired characteristics outlined earlier: Automation, Integration, Speed, Ease of Use, and Scalability. The goal was to determine which combination of interface and workflow structure best supports a fast, reliable, and user friendly documentation process for engineers at SanDisk.

  - **VSCode Extension**
    - **1. Automation:** The VSCode extension can fully automate the workflow from PlusCal verification to PDF generation. Once triggered, all processes run in sequence without requiring manual input. This makes it one of the most automated options available.
    - **2. Integration:** This option integrates directly into the engineer's existing environment, where TLA+, PlusCal, and PlantUML are already used. Users can generate diagrams and PDFs within the IDE, minimizing context switching and keeping all tools centralized. It therefore scores highest in integration.
    - **3. Speed:** Because the Rust backend runs locally and is invoked directly from VSCode, performance is nearly instant. The lack of intermediate manual steps avoids delays, earning a high score for speed.

- **4. Ease of Use:** This is the most user friendly approach, as engineers only need to use a single command or menu option within VSCode. No command line knowledge or file handling is required, making it intuitive even for new users.
- **5. Scalability:** The extension's interface can handle complex models seamlessly as the backend Rust executable does all heavy computation. As project size grows, the workflow remains consistent and stable.

➢ **CLI Tool**

- **1. Automation:** The CLI automates diagram and PDF generation, but only once the engineer manually executes the command. This introduces additional steps and limits full automation.
- **2. Integration:** Since the CLI runs outside VSCode, it fragments the workflow. Users must manually export files or re run commands between tools, reducing cohesion with the existing TLA+ setup.
- **3. Speed:** The core Rust engine performs equally fast as in the VSCode extension. However, manual context switching between the IDE and terminal slows practical workflow speed.
- **4. Ease of Use:** The CLI is less intuitive, requiring engineers to remember commands, parameters, and paths. It demands basic command line familiarity, which may increase user friction.
- **5. Scalability:** The CLI handles large models efficiently, but managing outputs for multiple files or design versions manually can become cumbersome in larger projects.

➢ **Integrated Pipeline**

- **1. Automation:** This approach links all processes (PlusCal parsing, PlantUML diagram generation, LaTeX PDF compilation) into a single automated pipeline. Once initiated, no user intervention is needed, giving it the highest automation rating.
- **2. Integration:** The pipeline ensures consistent outputs between all components. Every stage communicates directly within the same execution flow, preventing mismatches between diagrams and verified models.
- **3. Speed:** Integrated pipelines eliminate unnecessary file I/O and intermediate execution, significantly improving end to end speed. It ensures verified documentation can be generated within seconds.
- **4. Ease of Use:** Since the entire process runs automatically, users interact with it only once (through the VSCode interface or a single command). This makes it extremely simple to use.
- **5. Scalability:** The pipeline can handle large or complex projects efficiently. Its self contained structure makes it stable and predictable, even for heavy workloads.

- **External Scripts**
  - **1. Automation:** External scripts can partially automate the workflow but still require the user to execute each stage manually (e.g. parsing, diagram generation, PDF compilation). This limits automation and increases room for user error.
  - **2. Integration:** This approach relies on loosely connected scripts across different environments, making integration weak. File dependencies and tool versions may vary, leading to inconsistencies.
  - **3. Speed:** Each manual execution introduces latency, and potential errors or file handling issues can slow down the overall process. It performs the slowest in real world usage.
  - **4. Ease of Use:** Managing multiple scripts and dependencies can be confusing, especially for non expert users. It requires technical familiarity with each tool involved.
  - **5. Scalability:** As the project grows in size or complexity, managing external scripts becomes increasingly impractical. Performance and reliability degrade with scale.

- **Chosen Approach:**

VSCode Extension with Integrated Pipeline

Upon assessing the various implementation strategies of the P2P2P tool, it becomes clear that a combination of a VSCode Extension and an Integrated Rust based Pipeline would provide the best outcome for user experience. A standalone CLI tool is fairly simple to implement and can permit power users to be flexible while still being easy to use as an implementation, however it lacks the user experience and requires engineers to handle the files and outputs. An even simpler implementation approach would be using external scripts, although this creates fragmentation, increases errors while decreasing efficiency, and overall does not provide a reliable user experience. Ultimately, a combination of VSCode Extension and Integrated Automation Pipeline would produce slightly more complexity in development but would result in a seamless user experience while offering fully automated and reproducible back end processes to ensure correctness and efficiency.

Comparison Chart:

Alternative	Automation	Integration	Speed	Ease of Use	Scalability	Sponsor's Preference
VSCode Extension	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
CLI Tool	⭐⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐	⭐⭐⭐	⭐⭐⭐

Integrated Pipeline	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
External Scripts	⭐⭐	⭐	⭐⭐	⭐	⭐⭐	⭐

Table 3: Comparing performance of alternatives for Efficient Workflow in various categories

❖ **Proving Feasibility:**

To show that our proposed automated workflow is feasible, we conducted an initial review of the tools involved: VSCode, the TLA+ PlusCal extension, PlantUML, and LaTeX. Based on their capabilities and compatibility, it is clear that these tools can be integrated into a single workflow where verifying PlusCal models could trigger diagram and PDF generation:

1. Simple PlusCal code

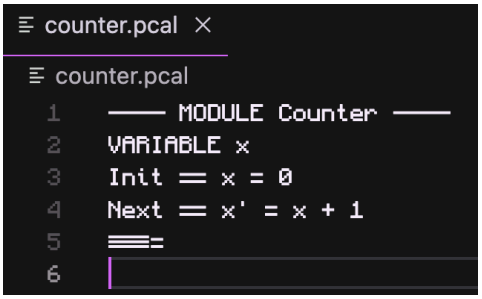


Figure 3: Example PlusCal code snippet

2. PlantUML file representing that code, and we can view it using the extension:

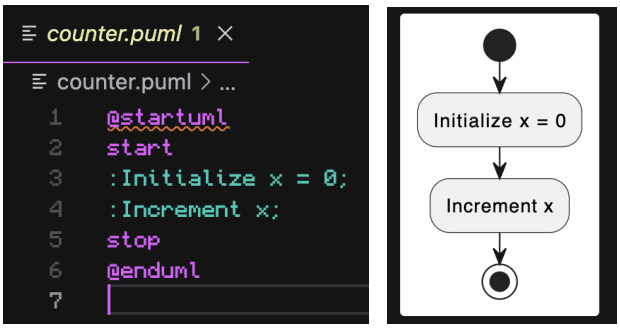


Figure 4: PlantUML example code with corresponding graphical representation

3. Now, using the PlantUML VSCode Extension, we can easily transform this diagram to whatever visual extension we prefer:

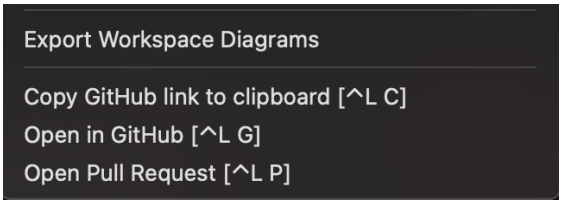


Figure 5: PlantUML export option

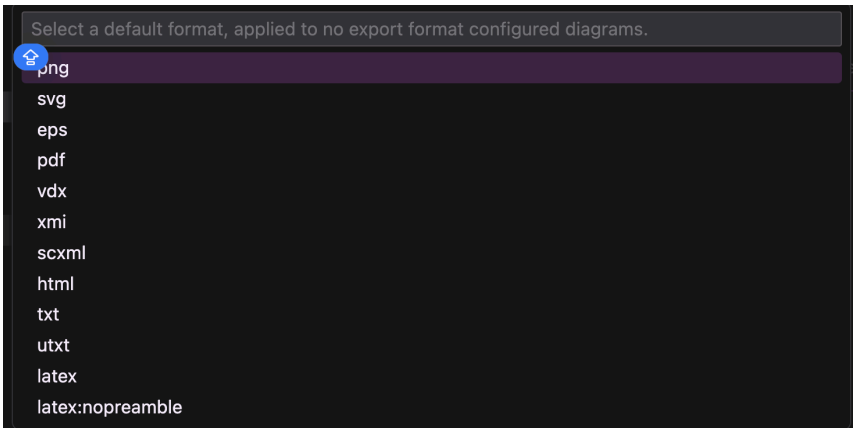


Figure 6: PlantUML export file types

4. Finally, the transformation has been done and we can see our diagram in the corresponding format, thanks to the PDF Viewer and LaTeX Workshop extensions:

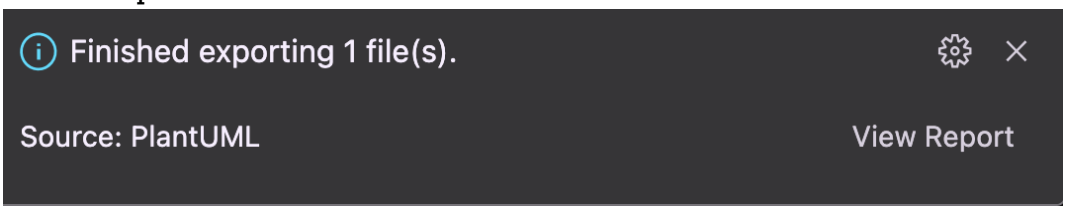


Figure 7: PlantUML export finishing

For further validation, we plan to perform small scale tests during the Technology Demo, starting with sample PlusCal models to confirm that each step of the workflow can execute correctly and in sequence.

3.5. Reliable Final Documentation

- **Introduce the Issue:**

  The final project deliverables must be professional and easy to reproduce. Manually combining diagrams and text into PDFs is both time consuming and prone to formatting differences. To meet the sponsor expectations and maintain quality across iterations, the team requires an automated and dependable process for converting the diagram outputs into polished PDF documents that the sponsor (SanDisk) engineers can utilize everyday.

- **Desired Characteristics:**

  - **Automation:** The entire build process should be able to be executed with a single command, eliminating the need for any type of manual compilation or directory management. This ensures efficiency and reduces human error during document generation.
  - **Integration:** Diagrams, figures, and written content should be fully connected so that any update made to a source file is automatically reflected in the final PDF. This guarantees up to date documentation without extra steps.
  - **Version Control:** All project assets, including LaTeX sources, diagrams, and outputs must be stored in Git for easy tracking, versioning, and collaborative editing. This approach allows seamless review, rollback, and reproducibility.
  - **Cross Platform:** The build system should function the same on Windows, macOS, and Linux to support the diverse setups of all team members. It should need little configuration and avoid platform specific issues.
  - **Ease of Learning:** Team members should be able to understand and use the workflow quickly. Clear documentation will help new or existing contributors to P2P2P seamlessly change any existing code without delay.
  - **Styling Control:** The generated PDFs must have a clean, professional appearance with consistent layout across all deliverables, ensuring a cohesive presentation to sponsors and faculty.
  - **Build Speed:** Document builds should be complete in under a minute, enabling fast iteration and reducing downtime during development and testing.
  - **Consistency:** Every team member's output should match exactly, ensuring the final deliverables appear uniform and meet the same standard.

- **Alternatives:**

  - Use LaTeX as the core renderer for the final PDF outputs:
    - LaTeX will serve as the core formatting pipeline and layout system for all project documentation. Its control over structure, citations,

and visual consistency makes it great for producing professional PDFs. LaTeX makes sure that every deliverable maintains the same standardized appearance, supports automated numbering for sections and figures, and integrates smoothly with version control systems.

- ➢ Use Pandoc to convert Markdown/PlantUML to PDF:
  - Before the final compilation, Pandoc will serve as a dynamic translation layer, converting the embedded PlantUML outputs and Markdown material into LaTeX compatible files. This enables the team to produce PDFs with LaTeX level quality while writing documentation in lightweight Markdown. Additionally, Pandoc's automation increases repeatability across settings and decreases the amount of manual formatting work.
- ➢ Use PlantUML's built in export to PDF for diagrams, then manually compile into the reports:
  - PlantUML will generate precise, up to date diagrams directly from text based models. These diagrams **CAN** be exported as PDFs and added into the main LaTeX report to ensure high quality resolution and scaling consistency. For this reason we felt like we should give it some thought as an option. While this step involves manual inclusion, it guarantees that each diagram remains accurate and visually aligned with the rest of the document, ensuring a clean, professional finish in sponsor facing deliverables.

❖ **Analysis:**

- ➢ LaTeX is widely used in the technical, and research fields. It offers professional grade document formatting. In order to ensure the same format across all sponsor ready reports, it offers unique control over layout, equations, tables, and references. It has a higher learning curve, nevertheless, we are being trained to understand its syntax and compilation procedure. Despite the high learning curve, it is a solid long term option for documentation due to its accuracy, dependability, and compatibility with version control.
- ➢ By instantly converting Markdown or other forms into PDFs or LaTeX, Pandoc does give a quicker and easier way to create documents. It is simple to set up and functions well for short texts and general reports. However, complicated formatting, cross references, or embedded diagrams can make Pandoc's automation problematic. Pandoc by itself might not offer the necessary control or visual quality because SanDisk's documentation frequently calls for extremely detailed and organized layouts.
- ➢ PlantUML is great at turning text based descriptions into UML style diagrams. It is perfect for quickly generating graphic models because of its integrated export feature, which enables these diagrams to be generated instantly as PDF, PNG, or SVG files. Nevertheless, this export feature is restricted to just the diagrams and does not offer professional

formatting, text integration, or full page layout. Therefore, rather than acting as a comprehensive documentation system, PlantUML will be utilized as a supporting tool inside the documentation pipeline, creating diagrams that are then integrated into LaTeX reports.

- **Chosen Approach:**

Use LaTeX as the core renderer for the final PDF outputs

That is why we believe that the best tool for producing expert PDF outputs that also follows accepted technical documentation standards was determined to be **LaTeX**. While HTML and Markdown to PDF renderers can produce PDFs much quicker, they provide less control over mathematical notation, citation style, and layout accuracy. LaTeX, on the other hand, offers extremely high levels of control over design and structure, guaranteeing uniform formatting throughout sections and figures. Additionally, it easily interfaces with version control systems, facilitating teamwork and reproducible builds. In comparison to Markdown based processes, LaTeX handles references, tables, and technical symbols better, according to reviews of the Pandoc and Overleaf documentation.

Comparison Chart:

Alternative	**LaTeX:**	**Pandoc:**	**PlantUML's BIE:**
Automation	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
Integration	⭐⭐⭐	⭐⭐	⭐⭐⭐⭐⭐
Version Control	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
Compatibility	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
Learning Curve	⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
Customization	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐⭐
Built Time (AVG)	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐

Table 4: Comparing alternatives for Reliable Final Documentation in various categories

- **Proving Feasibility:**

Viability of these options will be shown by embedding multiple PlantUML diagrams within a LaTeX template and exporting the document to PDF. Each output will be reviewed for consistent formatting, diagram clarity, and proper page alignment. Build times will also be measured to ensure the compilation finishes in under one minute, confirming the suitability for quick iteration. Lastly, version control tests through GitHub will make sure that LaTeX sources,

diagrams, and compiled PDFs can be tracked and updated reliably by all of the team members.

RESEARCH SCREENSHOTS:

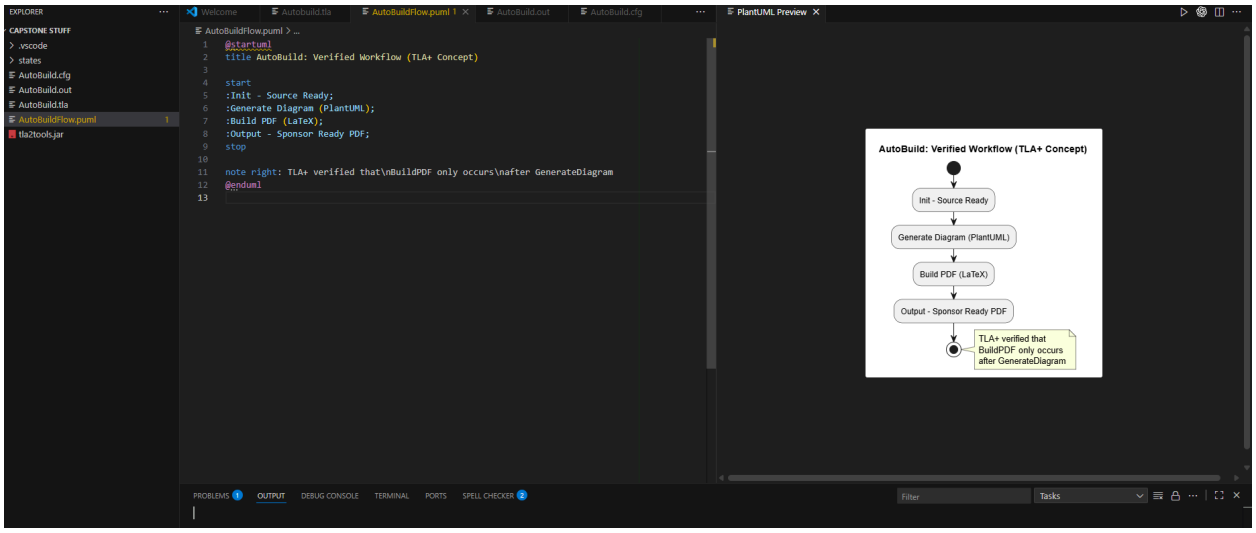


Figure 8: Using PlantUML to export the final documentation into a PDF.

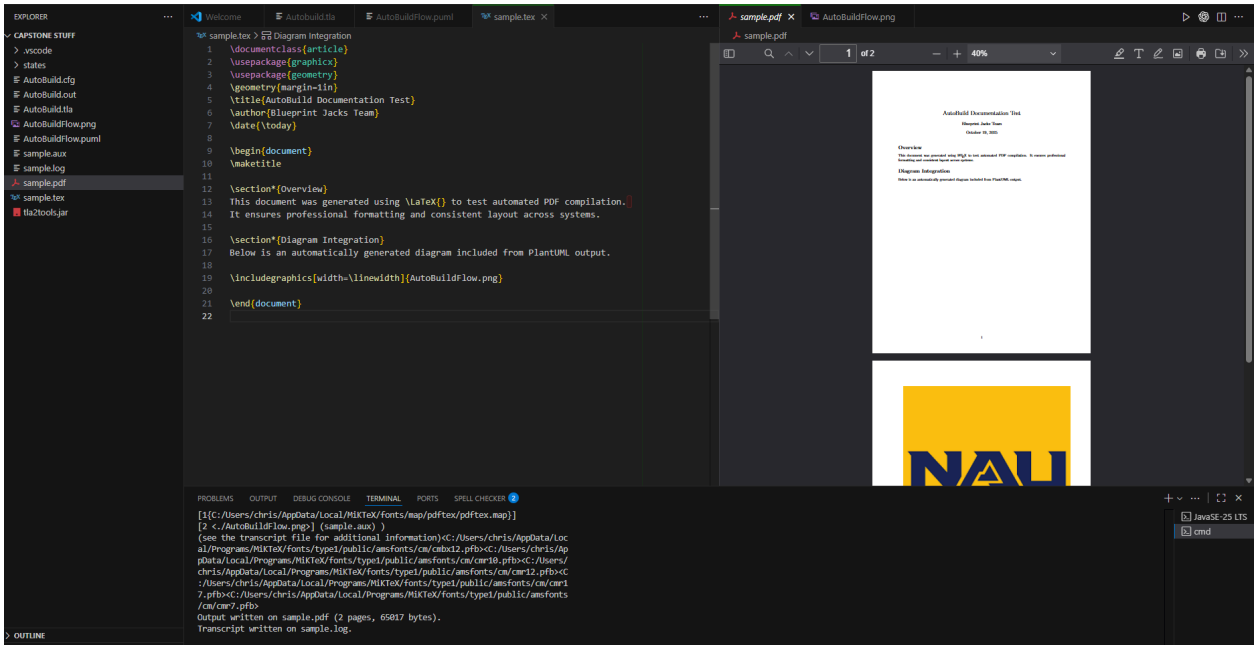


Figure 9: Using LaTeX to export the final documentation into a PDF.

4. Technology Integration

With the solutions we have come up with throughout this analysis, we can see how each of these solutions fit into our final product.
Through the first challenge of Automatic Diagram Generation, we cover the overall structure of the system and how it will all work by deciding on using Rust to write the program, which balances considerations about how quickly programs can be developed in Rust when compared to another, simpler programming language like Python, and that determines the overall structure and approach to development.

Then, when we decide on how to handle verification and when to generate models in the second challenge of Formal Verification and Accuracy, we get deeper into implementation details. We know how the program itself will be written, but we need to decide how that will that program fit into the overall structure of the TLC which will determine if a diagram gets generated at all, and we decided on checking the console output of the TLC would be the best idea given considerations like implementation time, maintenance, and the integration of that into the workflow.

From there, we need to focus on making the tool convenient to work with through our next challenge of Efficient Workflow. This focuses on automating as much of the work as possible and fitting neatly into the tools that the engineers already use for their development that is separate from diagrams, like their generally VSCode centered development. This means that based on a lot of factors, it makes the most sense for P2P2P to be a VSCode extension itself.

Lastly, for shipment to the shareholders or other engineers, we had to decide on final documentation preparation through the last challenge of Reliable Final Documentation. This means considering different ways to ship the product like which type of PDF software would be best and offer the most amount of features that suits both the project and the desire of the client. This is why we decided to go with LaTeX, which allows for a lot of customization in the final look of the PDFs produced with P2P2P as well as fitting nicely into version control systems.

All of these design decisions come together into the final system flow outlined below that shows how the program will lay out, translating PlusCal into PlantUML and finally into a PDF.

Technology integration system workflow:

PlusCal (VSCode)
↓
Parser/Converter (rust or python)
↓
PlantUML
↓
PDF Output

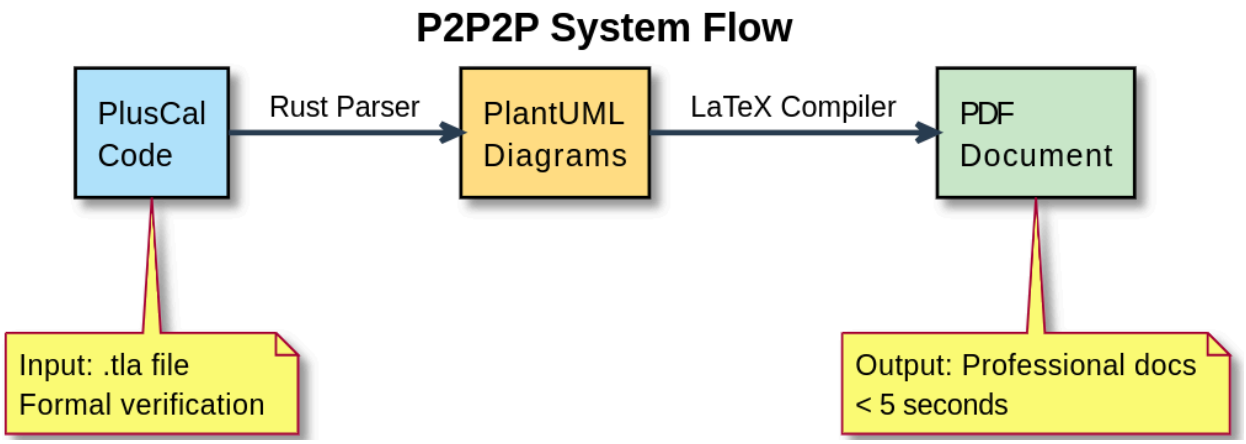


Figure 10: P2P2P System flow diagram showing complete program operation flow

5. Conclusion

The P2P2P project was born from a critical need at SanDisk: ensuring design correctness and documentation accuracy across millions of devices where even a single error can have global consequences. This tech feasibility analysis examined that challenge from every technical and organizational angle, showing how automation, accuracy, and efficiency can come together to improve the company's design verification process.

Throughout this report, our team identified four core technical challenges: automatic diagram generation, formal verification accuracy, workflow efficiency, and reliable documentation, and evaluated multiple alternatives for each. After thoughtful consideration, we have chosen a combination of technologies that best fits SanDisk's needs: Rust for safe and high-performance parsing, VSCode integration for streamlined work, PlantUML for accurate and flexible diagram generation, and LaTeX for providing high-quality, reproducible documentation. Collectively, these choices establish a complete, end-to-end solution to the company's pains by reducing manual involvement, removing inconsistencies, and ensuring each diagram aligns with the design that has been verified.

Going forward, the next steps are to create a working Rust prototype that translates PlusCal to PlantUML diagrams, to integrate this functionality in the VSCode environment, and to visualize the entire PlusCal to PDF automation pipeline. Our team is confident that, with the feasibility of each layer validated, and with a clear plan to implement further, P2P2P can proceed into development. This next stage will help bring SanDisk closer to a more reliable, efficient, and quality-oriented engineering workflow where any verified design can be generated clearly, confidently, and automatically.